

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch01_mathematics

AYuSh

Contents

Chapter 1: Mathematics for ML.....	3
------------------------------------	---

Chapter 1: Mathematics for ML

Every machine learning model you will ever discuss in an interview is, underneath the API, a mathematical object: a function with parameters, fit by optimization, judged by probability. Interviewers probe math not because they enjoy calculus, but because it is the fastest way to tell whether you *understand* models or merely *use* them. A candidate who can explain why eigenvalues matter for PCA, derive the gradient of logistic loss, or say precisely what a p-value is (and is not) signals depth that no amount of framework fluency can fake.

This chapter covers the four pillars — linear algebra, calculus, probability, and statistics — plus the three topics interviewers use as depth probes: hypothesis testing, MLE vs MAP, and information theory. If you are new, read it in order; everything later in the book stands on this chapter. If you are experienced, skim the headers and go straight to the Q&A at the end — if you can answer all of it out loud, move on.

Linear algebra

Vectors: the atoms of ML

A **vector** is an ordered list of numbers. That's it. The vector $x = [5.1, 3.5, 1.4, 0.2]$ might describe a flower by four measurements. In ML, every input — an image, a sentence embedding, a customer profile — becomes a vector, because vectors are things we can do arithmetic on.

Geometrically, a vector in $\mathbb{R}^d$ is a point (or an arrow from the origin) in d -dimensional space. Two operations carry almost all of ML:

The **dot product** $x \cdot w = \sum_{i=1}^d x_i w_i$ measures alignment. If the result is large and positive the vectors point the same way; zero means perpendicular (orthogonal); negative means opposing. A linear model's prediction $\hat{y} = w^T x + b$ is nothing more than "how aligned is this input with the direction w that the model learned to associate with the positive class?"

The **norm** $\|x\| = \sqrt{\sum_i x_i^2}$ measures length. Combining the two gives **cosine similarity**:

$$\cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|}$$

which strips away magnitude and keeps only direction — the standard similarity measure for text embeddings, where document length shouldn't matter but topic should.

Worked example. Let $x = [1, 2]$ and $y = [2, 1]$. Then $x \cdot y = 1 \cdot 2 + 2 \cdot 1 = 4$, $\|x\| = \|y\| = \sqrt{5}$, so $\cos(\theta) = 4/5 = 0.8$. The vectors are similar but not identical in direction — check: they are mirror images across the diagonal.

Matrices: functions, not tables

A **matrix** A of shape $m \times n$ is best understood not as a grid of numbers but as a *linear function* that maps vectors from $\mathbb{R}^n$ to $\mathbb{R}^m$ via $v \mapsto Av$. Every linear layer in a neural network is exactly this: `output = W @ input + b`. Matrix multiplication is function composition — $B(Av) = (BA)v$ — which is why a stack of linear layers *without nonlinearities* collapses into one linear layer: the product W_2W_1 is just another matrix. This single fact is the reason activation functions exist, and it is a classic interview question.

Key facts you should be able to state cold: matrix multiplication requires inner dimensions to match ($m \times n$ times $n \times p$ gives $m \times p$); it is associative but **not commutative** ($AB \neq BA$ in general); the transpose reverses order, $(AB)^T = B^T A^T$; and the inverse A^{-1} exists only for square matrices with nonzero determinant (equivalently: full rank, no zero eigenvalues, columns linearly independent — interviewers love asking for all the equivalent conditions).

Rank is the number of linearly independent columns (equivalently rows) — the true dimensionality of the space the matrix can reach. A rank-deficient design matrix X means some feature is a linear combination of others (perfect multicollinearity), and $X^T X$ becomes non-invertible — which is precisely why the normal equation for linear regression can fail and why Ridge regression adds λI to fix it. Chapter 5 builds on this.

Eigenvalues and eigenvectors: the directions a matrix cannot rotate

For a square matrix A , an **eigenvector** v is a special direction that A does not rotate — it only stretches or shrinks it:

$$Av = \lambda v$$

The stretch factor λ is the **eigenvalue**. Think of a matrix as a transformation of space: most directions get rotated somewhere else, but eigenvectors are the axes of the transformation, and eigenvalues tell you how much each axis is scaled.

Worked example. Let A be the diagonal matrix with entries 2 and 3 (rows (2, 0) and (0, 3)). Any vector along the x-axis satisfies $Av = 2v$, any vector along the y-axis satisfies $Av = 3v$. Eigenvalues are 2 and 3, eigenvectors are $[1, 0]$ and $[0, 1]$. For non-diagonal matrices the eigenvectors are not axis-aligned, but the idea is identical. To find eigenvalues by hand, solve the characteristic equation $\det(A - \lambda I) = 0$: for the matrix with rows (2, 1) and (1, 2), this is $(2 - \lambda)^2 - 1 = 0$, so $\lambda = 1$ or $\lambda = 3$ (with eigenvectors along $[1, -1]$ and $[1, 1]$ — check by multiplying).

Why ML interviews care:

PCA (Chapter 8) finds the eigenvectors of the data's covariance matrix. The eigenvector with the largest eigenvalue is the direction of maximum variance — the "principal component." Eigenvalues tell you how much variance each component explains, which is how you decide how many components to keep.

Optimization geometry. The eigenvalues of the Hessian (see calculus section) describe the curvature of the loss surface. A large ratio between the largest and smallest eigenvalue (the **condition number**) means a long, narrow valley — exactly the situation where gradient descent zig-zags and converges slowly, and the motivation for momentum and adaptive optimizers (Chapter 13).

Spectral properties of networks. Repeated multiplication by a weight matrix amplifies components along eigenvectors with $|\lambda| > 1$ and kills those with $|\lambda| < 1$ — the linear-algebra heart of exploding and vanishing gradients in RNNs (Chapter 15).

SVD: the decomposition that always exists

Eigendecomposition requires a square (and, to be well-behaved, symmetric) matrix. The **Singular Value Decomposition** works for *any* matrix, even rectangular:

$$A = U\Sigma V^T$$

where U ($m \times m$) and V ($n \times n$) are orthogonal matrices (their columns are orthonormal directions — pure rotations/reflections), and Σ is diagonal with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. Read it as: *any* linear map is a rotation, followed by axis-aligned scaling, followed by another rotation. The singular values are the scaling factors, and they equal the square roots of the eigenvalues of $A^T A$.

The interview-critical application is **low-rank approximation**: keep only the top k singular values and the corresponding columns of U and V :

$$A \approx U_k \Sigma_k V_k^T$$

The Eckart-Young theorem says this is the *best* rank- k approximation of A in the least-squares sense. This is the math behind classic recommender-system matrix factorization (Chapter 30), latent semantic analysis in NLP, model-weight compression, and — because LoRA (Chapter 21) parameterizes weight *updates* as a low-rank product BA — the fine-tuning method you will most likely be asked about in an LLM interview. PCA itself is SVD applied to mean-centered data.

Other decompositions worth naming in an interview: **Cholesky** ($A = LL^T$ for symmetric positive-definite matrices; fast solving of linear systems, sampling from multivariate Gaussians) and **QR** (numerically stable least squares). You rarely need their mechanics — you need to know *when* they apply.

Calculus

Derivatives and gradients: which way is down

The derivative $f'(x)$ is the local slope: how much f changes per unit change in x . Machine learning needs exactly one big idea from calculus: **to make a loss smaller, move parameters in the direction opposite the slope**. Everything else is bookkeeping for functions of many variables.

When a function takes many inputs, $f(w_1, \dots, w_d)$, the **partial derivative** $\partial f / \partial w_i$ is the slope with respect to one input, holding the others fixed. Collecting all partials into a vector gives the **gradient**:

$$\nabla_w f = \left[\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_d} \right]^T$$

The gradient points in the direction of *steepest increase*, and its norm says how steep that ascent is. Gradient descent therefore updates $w \leftarrow w - \eta \nabla_w \mathcal{L}$, stepping downhill with step size η (Chapter 13 covers what happens when η is wrong).

Worked example. $f(w_1, w_2) = w_1^2 + 3w_1w_2$. Then $\partial f / \partial w_1 = 2w_1 + 3w_2$ and $\partial f / \partial w_2 = 3w_1$. At the point $(1, 2)$: $\nabla f = [8, 3]^T$. To decrease f from that point, step along $[-8, -3]$.

The chain rule: the engine of backpropagation

Deep networks are compositions of functions: $\mathcal{L} = f_3(f_2(f_1(x; w_1); w_2); w_3)$. The **chain rule** says the derivative of a composition is the product of the derivatives of the pieces:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Backpropagation (Chapter 12 derives it fully) is *just* the chain rule applied systematically, layer by layer, reusing intermediate results so nothing is computed twice. Two facts to internalize now: the chain rule multiplies local derivatives, so a chain of numbers slightly below 1 shrinks exponentially (vanishing gradients) and a chain slightly above 1 grows exponentially (exploding gradients); and computing the chain *backwards* (from loss to inputs) is cheap when the output is a scalar — one backward sweep gets every parameter's gradient at once.

Worked example. $\mathcal{L}(w) = (\sigma(wx) - y)^2$ with $x = 2$, $y = 1$, $w = 0$. Then $\sigma(0) = 0.5$. Chain: $\frac{d\mathcal{L}}{dw} = 2(\sigma(wx) - y) \cdot \sigma'(wx) \cdot x$. Since $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = 0.25$ at $z = 0$: $\frac{d\mathcal{L}}{dw} = 2(0.5 - 1)(0.25)(2) = -0.5$. The gradient is negative, so increasing w decreases the loss — which makes sense: we need $\sigma(2w)$ to move toward 1.

Jacobians and Hessians

When a function maps vectors to vectors, $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, its derivative is the **Jacobian** — the $m \times n$ matrix of all partial derivatives $J_{ij} = \partial f_i / \partial x_j$. Each layer of a network has a Jacobian;

backprop is a product of Jacobians (multiplied against a vector, which is why it's efficient — you never materialize the full matrices).

The **Hessian** is the matrix of second derivatives of a scalar function: $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. It describes *curvature*. At a critical point (zero gradient): all Hessian eigenvalues positive → local minimum; all negative → local maximum; mixed signs → saddle point. In high dimensions saddle points vastly outnumber local minima, which is part of why plain gradient descent works better in deep learning than early theory predicted — most zero-gradient traps are saddles you can escape, not minima you're stuck in.

The Hessian's eigenvalue spread (condition number, again) controls how hard the optimization is; Newton's method uses $H^{-1}\nabla f$ to rescale steps by curvature but is $O(d^2)$ memory and $O(d^3)$ compute per step — hopeless for a model with millions of parameters, which is why deep learning uses first-order optimizers with cheap curvature *approximations* like Adam's per-parameter second-moment estimates (Chapter 13).

Probability

The rules of uncertainty

Probability is the mathematics of consistent reasoning under uncertainty. Three rules generate everything else. **Non-negativity and normalization**: probabilities are in $[0, 1]$ and sum to 1 over all outcomes. The **sum rule** (marginalization): $P(A) = \sum_B P(A, B)$ — if you don't care about B , sum it out. The **product rule**: $P(A, B) = P(A | B)P(B)$ — a joint probability factors into a conditional times a marginal.

Conditional probability $P(A | B) = P(A, B)/P(B)$ is the probability of A *within the world where B happened*. Conditioning is re-normalizing to a smaller universe. Two events are **independent** when conditioning tells you nothing: $P(A | B) = P(A)$, equivalently $P(A, B) = P(A)P(B)$.

Bayes' theorem: inverting the question

Rearranging the product rule gives the most important equation in applied statistics:

$$P(H | D) = \frac{P(D | H)P(H)}{P(D)}$$

Read: the **posterior** probability of hypothesis H given data D equals the **likelihood** of the data under the hypothesis, times the **prior** probability of the hypothesis, normalized by the **evidence** $P(D)$. Bayes' theorem converts "how likely is this data if the hypothesis is true?" (easy to model) into "how likely is the hypothesis given this data?" (what you actually want).

Worked example — the one interviewers actually ask. A disease affects 1% of the population. A test has 95% sensitivity ($P(+ | \text{sick}) = 0.95$) and 90% specificity ($P(- | \text{healthy}) = 0.90$). You test positive. What is the probability you are sick?

$$P(\text{sick} | +) = \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.10 \times 0.99} = \frac{0.0095}{0.0095 + 0.099} \approx 0.088$$

About 9%, not 95%. The prior (1% base rate) dominates because false positives from the huge healthy population swamp true positives from the tiny sick population. The lesson — **never ignore the base rate** — is the core intuition behind why accuracy is a useless metric on imbalanced data (Chapter 10) and why Naive Bayes works the way it does (Chapter 6).

Distributions you must know

A **random variable** is a quantity whose value is uncertain; its **distribution** assigns probabilities to its possible values (a probability *mass* function for discrete variables, a *density* for continuous ones).

Bernoulli(p) — a single yes/no trial: $P(X = 1) = p$, mean p , variance $p(1 - p)$. The output of a binary classifier is a Bernoulli parameter; log loss is exactly the Bernoulli negative log-likelihood.

Binomial(n, p) — the number of successes in n independent Bernoulli trials: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$, mean np , variance $np(1 - p)$. This is the distribution of "how many of my 10,000 users clicked," which makes it the backbone of A/B test analysis.

Poisson(λ) — counts of rare events in a fixed window when events are independent: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, with mean *and* variance both λ (a fact interviewers check). It is the limit of Binomial(n, p) as $n \rightarrow \infty$, $p \rightarrow 0$ with $np = \lambda$ fixed. Used for arrival counts: requests per second, fraud events per day.

Gaussian $\mathcal{N}(\mu, \sigma^2)$ — the bell curve:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Why it is everywhere: (1) the Central Limit Theorem (below) makes sums of independent effects approximately Gaussian; (2) it is the maximum-entropy distribution for a fixed mean and variance — the "least assuming" choice; (3) assuming Gaussian noise in a regression model makes maximum likelihood *identical* to minimizing squared error, a derivation done in the MLE section below and asked verbatim in interviews. Remember the 68-95-99.7 rule: those are the probabilities of landing within 1, 2, and 3 standard deviations of the mean.

Worked examples with the distributions.

Binomial. A model serves 1,000 users; each independently converts with probability 0.1. Expected conversions: $np = 100$. Standard deviation: $\sqrt{np(1 - p)} = \sqrt{90} \approx 9.5$. So a day

with 80 conversions is about two standard deviations low — unusual but not impossible ($\approx 2.5\%$ of days by the normal approximation); a day with 50 is a six-sigma event, and you should suspect the pipeline, not chance. This "expected $\pm$ a few SDs" reflex is the fastest sanity check in applied ML.

Poisson. A fraud system sees on average $\lambda = 3$ fraudulent transactions per hour. Probability of a completely clean hour: $P(X = 0) = e^{-3} \approx 0.05$. Probability of 8 or more (an alarm threshold): small enough ($\sim 1\%$) that alerting on it has a tolerable false-alarm rate. Because Poisson variance equals its mean, a monitoring stream whose counts vary much more than their mean (**overdispersion**) violates the Poisson assumption — usually a sign of clustered or bursty events, and a favorite trick question.

Gaussian and z-scores. Model latency is roughly $\mathcal{N}(120, 15^2)$ milliseconds. What fraction of requests exceed 150 ms? Standardize: $z = (150 - 120)/15 = 2$, and $P(Z > 2) \approx 2.3\%$. Standardizing to z-scores — subtract the mean, divide by the SD — is the same operation as feature standardization in Chapter 9; it puts any Gaussian on the common $\mathcal{N}(0, 1)$ scale where tables and intuition apply.

Statistics

Describing data

The **mean** $\bar{x} = \frac{1}{n} \sum_i x_i$ is the balance point; it minimizes squared distance to the data but is dragged by outliers. The **median** (middle value) minimizes absolute distance and resists outliers — for skewed data like incomes or latencies, median (and percentiles like p95/p99) is the honest summary. The **mode** is the most frequent value. In a right-skewed distribution: mode < median < mean — a favorite quick interview check.

Variance measures spread: $\text{Var}(X) = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$. Its square root, the **standard deviation** σ , is in the same units as the data. When estimating from a sample, divide by $n - 1$ instead of n (**Bessel's correction**): the sample mean is itself fit to the data and "uses up" one degree of freedom, making raw $1/n$ variance systematically too small.

Covariance measures how two variables move together: $\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$. Its scale-free version is the **Pearson correlation** $\rho = \text{Cov}(X, Y)/(\sigma_X \sigma_Y) \in [-1, 1]$. Three traps to name in interviews: correlation measures only *linear* relationships (a perfect parabola can have $\rho = 0$); correlation is not causation (confounders, selection bias); and zero correlation does not imply independence (though independence does imply zero correlation). The covariance *matrix* of a dataset — $\Sigma = \frac{1}{n} X_c^T X_c$ for centered data — is the object PCA eigendecomposes.

Law of Large Numbers and the Central Limit Theorem

The **Law of Large Numbers** says the sample mean converges to the true mean as n grows: averages stabilize. It is why Monte Carlo methods work, why more data reduces estimation noise, and why your A/B test needs enough samples before its metric means anything.

The **Central Limit Theorem** is stronger and stranger: the *distribution* of the sample mean approaches a Gaussian, no matter what distribution the individual samples come from (as long as they are independent with finite variance):

$$\bar{X}_n \sim \mathcal{N}(\mu, \frac{\sigma^2}{n})$$

The spread of the sample mean — the **standard error** $\sigma/\sqrt{n}$ — shrinks with $\sqrt{n}$, which is the reason halving your error bars requires *four times* the data, and the mathematical foundation of every confidence interval and significance test in the next section. LLN says *where* the average goes; CLT says *how it fluctuates* on the way.

Hypothesis testing

The logic of a hypothesis test is proof by statistical contradiction. State a **null hypothesis** H_0 ("the new model is no better than the old one"). Assume it is true. Compute how surprising the observed data would be under that assumption. If the data is too surprising, reject H_0 .

The **p-value** is the probability, *assuming H_0 is true*, of observing data at least as extreme as what you saw. It is **not** the probability that H_0 is true — that misstatement is the single most common statistics error in interviews, and interviewers bait it deliberately. A p-value below a pre-chosen threshold α (conventionally 0.05) means "reject H_0 "; α is exactly the **Type I error** rate — the probability of rejecting a true null (false alarm). **Type II error** (β) is failing to reject a false null (missed detection), and **power** $= 1 - \beta$ is the probability of catching a real effect. Power grows with effect size and sample size; that is why you run a *power analysis before* an experiment to decide how much data you need.

The t-test compares means. The one-sample statistic is $t = (\bar{x} - \mu_0)/(s/\sqrt{n})$: how many standard errors is the sample mean from the hypothesized mean? The two-sample version compares two groups (e.g., treatment vs control conversion values). It assumes approximately normal sample means (CLT gives you this for large n) and, in the classic form, similar variances (Welch's variant relaxes this — say "Welch's t-test" in interviews when unsure).

The chi-square test handles categorical counts: $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$ over cells of a contingency table, where O is observed and E is the count expected under independence. Use it for "did signup rate differ across these five landing pages?" or goodness-of-fit checks.

Confidence intervals. A 95% CI for a mean is $\bar{x} \pm 1.96 \cdot s/\sqrt{n}$. Precise meaning (another bait question): if you repeated the experiment many times, 95% of the intervals constructed this way would contain the true value. It is a statement about *the procedure*, not "there's a 95% chance the truth is in this particular interval."

Power analysis, worked. How many users per arm to detect a lift from 10.0% to 10.8% conversion at $\alpha = 0.05$ (two-sided) with 80% power? The standard formula for two proportions is

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \cdot 2\bar{p}(1-\bar{p})}{(p_B - p_A)^2}$$

with $z_{0.975} = 1.96$, $z_{0.80} = 0.84$, $\bar{p} = 0.104$: $n \approx \frac{(2.8)^2 \times 2 \times 0.0932}{(0.008)^2} \approx 22,800$ per arm. This is why the 10,000-per-arm experiment in the code section below *correctly* fails to find the effect — it was never big enough to see it. Memorize the structure, not the constants: required n scales with variance and *inversely with the square* of the effect size — detecting an effect half as big costs four times the users.

A/B testing packages all of this: randomize users into control (A) and treatment (B), pre-register the metric and α , run the power analysis, collect data, run the test (t-test for continuous metrics, chi-square or two-proportion z-test for conversion rates). Practical failure modes interviewers probe: **peeking** (checking repeatedly and stopping when significant inflates false positives far above α); **multiple comparisons** (testing 20 metrics at $\alpha = 0.05$ expects one false positive — use Bonferroni or FDR corrections); **novelty effects** (users click new things briefly); and **interference** (one user's treatment affecting another's behavior, e.g., in marketplaces or social feeds). Chapter 27 revisits A/B testing for model deployment.

Maximum likelihood and MAP

MLE: let the data speak

You have data and a model with unknown parameters θ . **Maximum Likelihood Estimation** picks the θ that makes the observed data most probable:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} P(D|\theta) = \arg \max_{\theta} \prod_{i=1}^n P(x_i | \theta)$$

The product (from assuming independent samples) turns into a sum under a logarithm — which is monotonic, so maximizing log-likelihood is equivalent and numerically far saner:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log P(x_i | \theta)$$

Worked example (coin). Flip a coin 10 times, see 7 heads. Likelihood:

$L(p) = p^7(1-p)^3$. Log-likelihood: $\ell(p) = 7\log p + 3\log(1-p)$. Set the derivative to zero:

$7/p - 3/(1-p) = 0 \Rightarrow p = 0.7$. MLE just returns the empirical frequency — data speaks, nothing else does.

The derivation interviewers actually want. Assume regression targets are generated as $y_i = w^T x_i + \varepsilon_i$ with Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The log-likelihood of the data is

$$\ell(w) = \sum_i \log \mathcal{N}(y_i | w^T x_i, \sigma^2) = -\frac{1}{2\sigma^2} \sum_i (y_i - w^T x_i)^2 + \text{const}$$

Maximizing it is *exactly* minimizing the sum of squared errors. **MSE is not an arbitrary choice — it is Gaussian-noise MLE.** Likewise, cross-entropy loss is Bernoulli/categorical MLE. Nearly every loss function you will ever use is a negative log-likelihood in disguise; saying this unprompted is a strong interview signal.

MAP: data plus belief

MLE can overfit small samples (see 3 heads in 3 flips $\rightarrow \hat{p} = 1$, a certainty no reasonable person holds). **Maximum A Posteriori** estimation multiplies in a prior over parameters and maximizes the posterior:

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(D|\theta)P(\theta) = \arg \max_{\theta} [\log P(D|\theta) + \log P(\theta)]$$

The prior acts as a penalty on implausible parameters. The connection interviewers prize: **a Gaussian prior on weights gives L2 regularization (Ridge); a Laplace prior gives L1 (Lasso).** Take $P(w) = \mathcal{N}(0, \tau^2)$: $\log P(w) = -\|w\|^2/(2\tau^2) + \text{const}$, so MAP = MLE loss + $\lambda\|w\|^2$ with $\lambda = \sigma^2/\tau^2$. Regularization is not a hack — it is a prior belief that weights should be small. As $n \rightarrow \infty$ the likelihood term dominates and MAP converges to MLE: with enough data, the prior stops mattering.

Information theory

Entropy: the price of uncertainty

The **surprise** of an outcome with probability p is $-\log p$ — rare events are more surprising, certain events not at all. **Entropy** is expected surprise:

$$H(P) = -\sum_x P(x) \log P(x)$$

With log base 2, entropy is measured in bits and has an operational meaning: the minimum average number of bits needed to encode samples from P . A fair coin has $H = 1$ bit; a coin with $p = 0.99$ has $H \approx 0.08$ bits — nearly no information because outcomes are nearly certain. Uniform distributions maximize entropy; deterministic ones have zero. Decision trees (Chapter 6) split on features that maximally *reduce* entropy of the label — information gain.

Cross-entropy and KL divergence

Cross-entropy is the expected surprise when reality follows P but you encode using model Q :

$$H(P, Q) = -\sum_x P(x) \log Q(x)$$

Training a classifier minimizes exactly this, with P the one-hot true label and Q the softmax output — which collapses to $-\log Q(\text{true class})$: penalize the model by how surprised it is at the truth.

KL divergence is the *excess* cost of using the wrong distribution:

$$D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P)$$

Properties to state in interviews: $D_{KL} \geq 0$, zero iff $P = Q$; and it is **asymmetric** — $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$, so it is not a distance metric. Since $H(P)$ is fixed by the data, minimizing cross-entropy is minimizing KL divergence from the model to the truth. KL appears throughout the modern stack: the regularizer in VAEs (Chapter 18), the policy constraint in PPO and RLHF (Chapters 21, 32), and knowledge distillation losses.

Mutual information measures how much knowing one variable reduces uncertainty about another: $I(X; Y) = H(X) - H(X | Y) = D_{KL}(P(X, Y) \| P(X)P(Y))$. It is zero iff X and Y are independent, and unlike correlation it captures *nonlinear* dependence — the principled version of "are these two features related?", used in feature selection (Chapter 9).

Code implementations

Every listing below was executed as printed; outputs are real. Interviewers frequently ask for exactly these implementations "from scratch, no sklearn."

PCA via eigendecomposition — and via SVD

```
import numpy as np
rng = np.random.default_rng(42)

X = rng.normal(size=(200, 2)) @ np.array([[3.0, 0.0], [1.0, 0.5]]) # correlated data
Xc = X - X.mean(axis=0) # 1. center the data
cov = (Xc.T @ Xc) / (len(Xc) - 1) # 2. covariance matrix (d x d)
eigvals, eigvecs = np.linalg.eigh(cov) # 3. eigendecomposition (symmetric -> eigh)
order = np.argsort(eigvals)[::-1] # 4. sort by descending eigenvalue
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
explained = eigvals / eigvals.sum()
Z = Xc @ eigvecs[:, :1] # 5. project onto top-1 component
print("eigenvalues:", np.round(eigvals, 3))
print("explained variance ratio:", np.round(explained, 3))
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
print("PC1 from eigh:", np.round(eigvecs[:, 0], 3), "| PC1 from SVD:", np.round(Vt[0], 3))
```

Output:

```
eigenvalues: [9.992 0.184]
explained variance ratio: [0.982 0.018]
PC1 from eigh: [-0.999 -0.051] | PC1 from SVD: [0.999 0.051]
```

Line by line: centering matters because PCA finds directions of variance *around the mean* — skip it and the first component points at the mean instead of along the data. We use `eigh` (not `eig`) because the covariance matrix is symmetric, which guarantees real eigenvalues and lets NumPy use a faster, stabler algorithm. The projection `Xc @ eigvecs[:, :1]` compresses 2-D data to 1-D while keeping 98.2% of the variance. The SVD route gives the same component — with an arbitrary sign flip, which is expected: if v is an eigenvector so is $-v$, and mentioning that sign ambiguity in an interview shows real fluency.

Gradient descent = Gaussian MLE, verified against the normal equation

```
import numpy as np
rng = np.random.default_rng(0)
n, d = 500, 3
X = rng.normal(size=(n, d))
true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + rng.normal(scale=0.3, size=n)

w, eta = np.zeros(d), 0.1
for step in range(200):
    grad = -2 / n * X.T @ (y - X @ w)  # d(MSE)/dw, derived via the chain rule
    w -= eta * grad
w_closed = np.linalg.solve(X.T @ X, X.T @ y)  # normal equation
print("gradient descent w:", np.round(w, 3))
print("normal equation w :", np.round(w_closed, 3))
```

Output:

```
gradient descent w: [ 2.015 -0.998  0.519]
normal equation w : [ 2.015 -0.998  0.519]
```

The gradient line is the chain rule from this chapter applied to $\mathcal{L}(w) = \frac{1}{n} \|y - Xw\|^2$. Note `np.linalg.solve` rather than explicitly inverting $X^T X$ — inverting is slower and numerically worse, and saying so is an easy interview point. Both estimates recover `true_w` up to noise, and they agree to three decimals: the iterative and closed-form solutions of the same convex problem.

MLE for a coin, numerically

```
import numpy as np
flips = np.array([1,1,1,0,1,1,0,1,0,1])  # 7 heads out of 10
ps = np.linspace(0.01, 0.99, 99)
```

```
loglik = flips.sum()*np.log(ps) + (len(flips)-flips.sum())*np.log(1-ps)
print("MLE p-hat:", np.round(ps[np.argmax(loglik)], 2))
```

Output:

```
MLE p-hat: 0.7
```

The grid search lands exactly on the calculus answer 7/10 from the worked example — a useful sanity-check pattern: when you derive an estimator by hand, verify it numerically.

Entropy, cross-entropy, and KL divergence from scratch

```
import numpy as np
def entropy(p):
    p = np.asarray(p); return -np.sum(p * np.log2(p, where=p>0))
def cross_entropy(p, q):
    p, q = np.asarray(p), np.asarray(q); return -np.sum(p*np.log2(q, where=q>0))
def kl(p, q):
    return cross_entropy(p, q) - entropy(p)

P = [0.5, 0.5]; Q = [0.9, 0.1]
print("H(P) =", entropy(P), "bits | H(P,Q) =", round(cross_entropy(P,Q),3),
      "| KL(P||Q) =", round(kl(P,Q),3))
print("KL(Q||P) =", round(kl(Q,P),3), " (asymmetric!)")
```

Output:

```
H(P) = 1.0 bits | H(P,Q) = 1.737 | KL(P||Q) = 0.737
KL(Q||P) = 0.531 (asymmetric!)
```

The `where=p>0` guard implements the convention $0 \log 0 = 0$ without NaNs. The two KL values differ — the asymmetry stated in the theory section, demonstrated in four lines.

An A/B test from scratch — including its failure

```
import numpy as np
from math import erf, sqrt
rng = np.random.default_rng(7)
nA, nB = 10_000, 10_000
conv_A = rng.binomial(1, 0.100, nA) # control: 10.0% true rate
conv_B = rng.binomial(1, 0.108, nB) # treatment: 10.8% true rate
pA, pB = conv_A.mean(), conv_B.mean()
p_pool = (conv_A.sum() + conv_B.sum()) / (nA + nB)
se = np.sqrt(p_pool * (1 - p_pool) * (1/nA + 1/nB))
z = (pB - pA) / se
p_value = 2 * (1 - 0.5 * (1 + erf(abs(z) / sqrt(2)))) # two-sided, normal CDF
ci = (pB - pA - 1.96*se, pB - pA + 1.96*se)
print(f"control {pA:.4f}, treatment {pB:.4f}, lift {pB-pA:+.4f}")
print(f"z = {z:.3f}, p-value = {p_value:.4f}")
print(f"95% CI for lift: [{ci[0]:+.4f}, {ci[1]:+.4f}]")
```

Output:

```
control 0.1042, treatment 0.1059, lift +0.0017
z = 0.392, p-value = 0.6950
95% CI for lift: [-0.0068, +0.0102]
```

Read this output carefully, because it is the most instructive failure in the chapter: the treatment *really is* better (10.8% vs 10.0% by construction), yet the test says "not significant" ($p = 0.70$). Nothing is wrong with the code — the experiment is **under-powered**. Detecting a 0.8-point lift on a 10% base rate at 80% power needs roughly 30,000+ users per arm, not 10,000. This is exactly the scenario behind the interview question "your A/B test shows no effect — what do you conclude?" The correct answer: *absence of evidence is not evidence of absence; check the power and the CI* — and indeed the confidence interval $[-0.007, +0.010]$ happily contains the true lift of $+0.008$.

The CLT, watched directly

```
import numpy as np
rng = np.random.default_rng(7)
raw = rng.exponential(scale=1.0, size=(100_000,)) # very non-Gaussian
means = rng.exponential(scale=1.0, size=(100_000, 50)).mean(axis=1)
print("raw skewness :", round(float(((raw-raw.mean())**3).mean() / raw.std()**3), 2))
print("means skewness:", round(float(((means-means.mean())**3).mean() /
means.std()**3), 2))
print("std of means :", round(float(means.std()), 4), "~ sigma/sqrt(n) =", round(1/
np.sqrt(50), 4))
```

Output:

```
raw skewness : 1.97
means skewness: 0.3
std of means : 0.1411 ~ sigma/sqrt(n) = 0.1414
```

Exponential samples are heavily skewed (1.97), but averages of just 50 of them are already nearly symmetric (0.3) — the CLT in action — and their spread matches the predicted standard error $\sigma/\sqrt{n}$ to three decimals.

Pitfalls, comparisons and practical tips

Confusion	Resolution
Eigendecomposition vs SVD	Eigen: square matrices, $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. SVD: any matrix, $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$. For symmetric PSD matrices (covariance), they coincide. PCA can be computed either way; SVD on centered data is more numerically stable.
Gradient vs Jacobian vs Hessian	Gradient: scalar function, vector of first derivatives. Jacobian: vector function, matrix of first derivatives. Hessian: scalar function, matrix of <i>second</i> derivatives (curvature).

Confusion	Resolution
Likelihood vs probability	Probability: fixed parameters, varying data. Likelihood: fixed data, varying parameters. Same formula, opposite viewpoint.
p-value	It is $P(\text{data at least this extreme} \mid H_0)$ — not $P(H_0 \mid \text{data})$, not the probability the result is a fluke, not effect size.
Confidence interval	A statement about the procedure's long-run coverage — not "95% chance the truth is in this interval."
MLE vs MAP	MAP = MLE + prior. Gaussian prior $\rightarrow$ L2 penalty; Laplace prior $\rightarrow$ L1. They converge as $n \rightarrow \infty$.
Cross-entropy vs KL	$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$. Minimizing one minimizes the other when P is fixed (labels).
Correlation vs independence	Independence $\implies$ zero correlation. Zero correlation $\nRightarrow$ independence (nonlinear dependence invisible to Pearson). Mutual information catches it.
Variance $1/n$ vs $1/(n-1)$	Sample variance uses $n-1$ (Bessel) because the sample mean consumed one degree of freedom.

Practical tips: when asked to derive anything, start by *defining every symbol* — interviewers grade setup as much as algebra. When numbers are involved (Bayes, tests), write the base rates first. And when you finish a derivation, sanity-check limiting cases out loud ($n \rightarrow \infty$, $p \rightarrow 0$, $\lambda \rightarrow 0$) — it catches sign errors and reads as maturity.

Interview questions and answers

Conceptual

Q1. What are eigenvalues and eigenvectors, intuitively, and where do they show up in ML?

An eigenvector of a matrix is a direction the matrix doesn't rotate — it only scales it, and the eigenvalue is that scale factor. They are the natural axes of a linear transformation. In ML: PCA takes the eigenvectors of the covariance matrix as directions of maximal variance; Hessian eigenvalues describe loss-surface curvature and condition optimization; and powers of weight matrices amplify or kill signal along eigen-directions, explaining exploding/vanishing gradients in RNNs.

Interviewers listen for: the geometric one-liner plus at least two concrete ML applications.

Q2. Why does PCA use the covariance matrix specifically?

PCA seeks the direction $\mathbf{v}$ (unit norm) maximizing the variance of the projected data $\text{Var}(X\mathbf{v}) = \mathbf{v}^T \Sigma \mathbf{v}$, where Σ is the covariance matrix. Maximizing $\mathbf{v}^T \Sigma \mathbf{v}$ subject to $\|\mathbf{v}\| = 1$ is solved (via a Lagrange multiplier) exactly by the top eigenvector of Σ , with the eigenvalue equal to the variance captured. So the covariance matrix isn't a choice — it falls out of the objective "find high-variance directions."

Q3. What is the difference between eigendecomposition and SVD, and when would you use each?

Eigendecomposition applies to square matrices and can involve complex values unless the matrix is symmetric; SVD applies to any matrix and always exists with real non-negative singular values. For a symmetric PSD matrix like a covariance, the two coincide. Use SVD for rectangular data matrices, low-rank approximation (Eckart-Young optimality), and numerical stability; use eigendecomposition when the object of interest is inherently square and symmetric.

Q4. Why can't a stack of linear layers without activations learn anything a single layer can't?

Because matrix multiplication composes into a matrix: $W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$, which is a single linear map. Depth without nonlinearity adds parameters but no expressive power. Nonlinear activations break the composition, letting depth build genuinely new function classes.

Q5. What does the condition number of a matrix tell you, and why should someone training models care?

It's the ratio of largest to smallest singular value (for the Hessian: eigenvalue). A large condition number means the loss surface is a long narrow valley: the gradient points mostly across the valley rather than along it, so gradient descent zig-zags and you must use a learning rate small enough for the steepest direction, making progress in the flattest direction glacial. This motivates momentum, Adam, second-order approximations, feature standardization, and batch norm.

Q6. Explain the difference between the Law of Large Numbers and the Central Limit Theorem.

LLN: the sample mean converges to the true mean — it tells you *where* averages go, but nothing about fluctuation. CLT: the distribution of the sample mean approaches a Gaussian with standard deviation $\sigma/\sqrt{n}$, regardless of the underlying distribution (finite variance, independence) — it tells you the *shape and size* of the remaining error, enabling confidence intervals and tests.

Q7. What is entropy, and what does it mean for a distribution to have high or low entropy?

Entropy is expected surprise, $H = -\sum p \log p$ — operationally, the minimum average bits to encode outcomes. Uniform distributions maximize it (nothing is predictable); deterministic ones have zero. In ML it appears as the impurity measure in decision trees, the target of label smoothing, and the exploration bonus in RL. A softmax output with high entropy means the model is unsure.

Q8. Why is KL divergence not a distance metric, and does it matter?

It's asymmetric ($D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$) and violates the triangle inequality. It matters practically: minimizing $D_{KL}(P\|Q)$ over Q (forward KL, as cross-entropy training does) is mean-seeking — Q spreads to cover all of P 's mass; minimizing $D_{KL}(Q\|P)$ (reverse KL, as in variational inference) is mode-seeking — Q locks onto one mode. Same "distance," qualitatively different fitted models.

Mathematical / derivation

Q9. Derive why minimizing MSE is equivalent to maximum likelihood under Gaussian noise.

Assume $y_i = f_w(x_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d. Then

$P(y_i | x_i, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-(y_i - f_w(x_i))^2 / 2\sigma^2)$. Log-likelihood:

$\ell(w) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - f_w(x_i))^2$. The first term is constant in w , so

maximizing ℓ is minimizing $\sum_i (y_i - f_w(x_i))^2$ — the sum of squared errors.

Interviewers listen for: stating the noise assumption up front, and noting the constant term drops.

Q10. Show that MAP with a Gaussian prior gives L2 regularization.

MAP maximizes $\log P(D | w) + \log P(w)$. With prior $w \sim \mathcal{N}(0, \tau^2 I)$:

$\log P(w) = -\frac{\|w\|^2}{2\tau^2} + c$. So MAP minimizes $\text{NLL}(w) + \frac{1}{2\tau^2} \|w\|^2$ — squared-error (or any) loss plus an L2 penalty with $\lambda = \sigma^2 / \tau^2$ for Gaussian likelihood. Tighter prior (small τ) $\rightarrow$ bigger $\lambda \rightarrow$ stronger shrinkage. A Laplace prior $P(w) \propto e^{-|w|/b}$ gives the L1 penalty the same way.

Q11. Derive the MLE for the parameter of a Bernoulli distribution.

Given k successes in n trials, $\ell(p) = k \log p + (n - k) \log(1 - p)$. Setting

$\ell'(p) = k/p - (n - k)/(1 - p) = 0$ gives $k(1 - p) = (n - k)p$, so $\hat{p} = k/n$: the sample frequency. Check the second derivative is negative (it is, for $p \in (0, 1)$), confirming a maximum.

Q12. A test for a condition with 2% prevalence has 90% sensitivity and 95% specificity. Compute the probability that a person testing positive actually has the condition.

$P(\text{sick} | +) = \frac{0.90 \times 0.02}{0.90 \times 0.02 + 0.05 \times 0.98} = \frac{0.018}{0.018 + 0.049} = \frac{0.018}{0.067} \approx 0.27$. Despite an accurate-sounding test, only $\sim 27\%$ of positives are true — the 98% healthy majority generates most of the positives. Always compute the denominator as (true positives + false positives).

Q13. Compute the entropy of a fair die, and of a die that always rolls 6.

Fair: $H = -\sum_{i=1}^6 \frac{1}{6} \log_2 \frac{1}{6} = \log_2 6 \approx 2.585$ bits — the maximum for 6 outcomes.

Deterministic: $H = -1 \log_2 1 = 0$ bits. Entropy measures spread of probability, not number of faces.

Q14. Why does sample variance divide by $n-1$?

Because deviations are measured from the *sample* mean, which was itself chosen to minimize those squared deviations — so $\sum (x_i - \bar{x})^2$ is systematically smaller than $\sum (x_i - \mu)^2$. Taking expectations shows $\mathbb{E}[\sum (x_i - \bar{x})^2] = (n-1)\sigma^2$, so dividing by $n-1$ makes the estimator unbiased. One degree of freedom was spent estimating the mean.

Q15. What is the gradient of $f(w) = \|y - Xw\|^2$, and what do you get when you set it to zero?

Expand: $f = (y - Xw)^T (y - Xw)$. Gradient: $\nabla_w f = -2X^T(y - Xw)$. Setting to zero: $X^T X w = X^T y$ — the normal equation, giving $\hat{w} = (X^T X)^{-1} X^T y$ when $X^T X$ is invertible (full column rank). If features are collinear it isn't; Ridge adds λI making $X^T X + \lambda I$ always invertible.

Statistical reasoning**Q16. Define p-value precisely. Your A/B test gives $p = 0.03$ — what can and can't you say?**

The p-value is the probability, computed assuming the null hypothesis is true, of data at least as extreme as observed. With $p = 0.03$ you can say: "if there were truly no effect, results this extreme would occur 3% of the time; at $\alpha = 0.05$ we reject the null." You cannot say: there's a 97% chance the treatment works, or the effect is large, or the finding will replicate. Effect size comes from the estimate and CI, not the p-value.

Q17. What are Type I and Type II errors, and how do they trade off?

Type I: rejecting a true null (false positive), rate α . Type II: failing to reject a false null (missed effect), rate β ; power = $1 - \beta$. Lowering α (stricter threshold) raises β for fixed data. You escape the tradeoff with more samples or bigger effects. Which error is worse is a product question: flagging fraud (Type I annoys users) vs missing fraud (Type II loses money).

Q18. Your A/B test shows no significant difference. The PM says "so the feature has no effect." How do you respond?

Non-significance is not evidence of no effect — the test may be under-powered. Check: (1) the confidence interval — if it includes effect sizes that would matter for the business, the test is inconclusive, not negative; (2) the pre-experiment power analysis — did we have the sample size to detect the minimum effect we care about? (3) data quality — randomization balance, metric definition. The chapter's Listing 5 shows a genuinely better treatment failing to reach significance at $n=10,000/\text{arm}$.

Q19. Why is peeking at an A/B test repeatedly and stopping when it hits significance a problem?

Each look is another chance for noise to cross the threshold; the actual false-positive rate across many peeks far exceeds the nominal α (with enough looks a null result will eventually "reach" $p < 0.05$ by chance). Fixes: fix the horizon in advance, use sequential testing methods designed for continuous monitoring (alpha-spending, mSPRT), or Bayesian approaches with explicit stopping rules.

Q20. When would you use a chi-square test vs a t-test?

t-test: comparing means of a continuous metric between groups (revenue per user, session length), relying on CLT normality of means. Chi-square: association between categorical variables from count data (converted/not $\times$ variant A/B/C), comparing observed to expected counts. For a 2×2 conversion table, the chi-square test is equivalent to the two-proportion z-test squared.

Q21. Correlation between two features is 0. A colleague concludes they're unrelated. What's wrong?

Pearson correlation captures only linear relationships. $Y = X^2$ with symmetric X has exactly zero correlation but complete dependence. Check scatterplots, rank correlation (Spearman) for monotone-nonlinear cases, or mutual information for general dependence. Only independence guarantees zero correlation, not the converse.

Applied / scenario**Q22. You must reduce 10,000 features to a few hundred for a downstream model. Walk through using SVD/PCA, including the decisions involved.**

Standardize features first (PCA chases variance — unscaled features with big units dominate). Compute truncated SVD of the centered matrix (never form the $10k \times 10k$ covariance explicitly; use randomized/truncated solvers). Choose k by cumulative explained variance (e.g., 95%), the eigenvalue elbow, or — best — downstream validation performance. Caveats to volunteer: components are linear and may destroy interpretability; PCA is unsupervised, so top-variance directions aren't necessarily predictive; fit the PCA on training data only to avoid leakage.

Q23. Training loss decreases smoothly, then explodes to NaN. Which mathematical concepts from this chapter diagnose it?

Exploding gradients: the chain rule multiplies per-layer Jacobians, and if their spectral norms exceed 1 the product grows exponentially; a large learning rate then turns a big gradient into a divergent step (η too large relative to local curvature — largest Hessian eigenvalue). Numerically, e^z in softmax/sigmoid overflows for large logits. Fixes: gradient clipping, lower η or warmup, normalization layers, numerically stable log-sum-exp implementations.

Q24. Metrics from a small user segment are extremely volatile week to week. A stakeholder wants to act on the latest week's number. What do you tell them?

Small n means large standard error ($\sigma/\sqrt{n}$): the weekly figure is mostly noise around the true rate. Quantify it — show the CI around the weekly estimate; it will likely span the entire "movement" being reacted to. Recommend aggregating weeks, hierarchical/shrinkage estimates toward the global mean (the Bayesian view: small samples should borrow strength from priors), and defining a minimum sample size before the metric is reported.

Q25. Why does cross-entropy loss, rather than accuracy, get optimized during classifier training?

Accuracy is a step function of the parameters — zero gradient almost everywhere, so gradient descent can't use it. Cross-entropy is smooth, aligns with MLE of the categorical likelihood, and its gradient w.r.t. the logits (softmax — one-hot) is simple and well-scaled: confident wrong predictions get large gradients, near-correct ones small. You optimize the differentiable surrogate, then report the step-function business metric.

Q26. You fit $p = 1.0$ for "heads" after seeing 3 heads in 3 flips. What went wrong, and what's the fix?

That's the MLE doing what it does — matching the empirical frequency — with too little data. It assigns zero probability to tails, an infinitely confident claim from 3 observations (and it breaks anything downstream taking $\log(0)$). Fix with a prior: MAP with a Beta prior gives $\hat{p} = (k + \alpha - 1)/(n + \alpha + \beta - 2)$; Laplace smoothing ($\alpha = \beta = 2$ style "add-one") is the classic instance, e.g. $(3 + 1)/(3 + 2) = 0.8$. This is exactly why Naive Bayes uses smoothing.

Q27. Explain, using this chapter's tools, why "we tested 25 metrics and shipped because engagement was significant at $p = 0.04$ " is a mistake.

Multiple comparisons: under 25 true nulls at $\alpha = 0.05$, the expected number of false positives is 1.25 — a single $p = 0.04$ among 25 metrics is entirely consistent with no real effect anywhere ($P(\text{at least one sig}) = 1 - 0.95^{25} \approx 72\%$). Fixes: pre-register one primary metric; correct thresholds (Bonferroni α/m , or Benjamini-Hochberg FDR for many metrics); treat the others as exploratory, to be confirmed in a follow-up test.

Q28. What is mutual information, and when would you prefer it to correlation for feature selection?

$I(X; Y) = H(Y) - H(Y | X)$: how many bits knowing X removes from the uncertainty of Y ; zero iff independent. Prefer it when relationships may be nonlinear or non-monotonic (correlation misses them), or for categorical variables where Pearson doesn't apply. Costs: it needs density/histogram estimation (harder with little data), returns no direction/sign, and like all filter methods scores features individually — it can miss features useful only in combination.